## Name

login – Gives access to the system.

## Description

The *login* program is used at the beginning of each terminal session to identify the user attempting to log in to the system. *Login* is invoked by *getty*(M), after a login name has been typed in response to the "login:" message. *Login* prompts for the user's password, then turns off echoing (where possible) to prevent the password from appearing on the terminal screen when typed.

Once a password has been typed, *login* encrypts the password and compares it with the encrypted password in the user's password entry (see *passwd*(M)). If there is a match, the login is successful.

If password aging is in effect and the current password is out of date, the *passwd*(C) command is automatically invoked. In this case, the user must change the password, then attempt to log in again.

If the login is not completed within a certain period of time (e.g., one minute), *login* either returns control to *getty*, or silently disconnects the dial-up line.

After a successful login, *login* updates accounting files, displays a message about the existence of any mail, then executes the start-up profile file, **/etc/profile** if you are running **sh** or **vsh** or **/etc/cshrc** if you are running **csh**. Next, *login* runs the user's **.profile** file ( **sh** or **vsh**) or his **.cshrc** and **.login** files ( **csh**) in his home directory. (See *profile*(M).) *Login* then initializes the user and group IDs, and the working directory. Finally, it executes a command interpreter (usually *sh*(C)) according to specifications found in the **/etc/passwd** file.

At some installations, an option may be invoked that will require a second "external" password to be entered. This will occur only for dial-up connections, and will be prompted by the message "External security:". Both passwords are required for a successful login.

When the user's shell is finally invoked, argument 0 of the command interpreter is a dash (-) followed by the last component of the interpreter's pathname. The *environment* (see *environ*(M)) is initialized to:

HOME= *your-login-directory*

PATH=:/bin:/usr/bin

Also, the user's file creation mask is set to octal 022 (see *umask*(C)).

## Files

| | |
|---|---|
| /etc/utmp | Accounting file |
| /etc/wtmp | Accounting file |
| /usr/spool/mail/*your-name* | Mailbox for user *your-name* |
| /etc/motd | Message of the day |
| /etc/passwd | Password file |
| /etc/profile | System profile |
| $HOME/.profile | Personal profile |

## See Also

environ(M), getty(M) mail(C), newgrp(C), passwd(C), passwd(M), profile(M), sh(C), su(C), umask(C)

## Diagnostics

*Login incorrect*
The user name or the password is incorrect.

*No shell, cannot open password file, no directory*:
Your account has not been properly set up.

*Your password has expired. Choose a new one.*
Password aging is in effect and yours has expired.

## Name

lp, lp0, lp1, lp2, lpT0, lpT1, lpT2 – Printer device interface

## Description

Use the lp files to access the optional parallel ports of your computer. The files correspond as follows:

| File | Port |
|---|---|
| lp0 | parallel port 1 (non-interpretive mode) |
| lp1 | parallel port 2 (non-interpretive mode) |
| lp2 | parallel port of a monochrome/parallel adapter (non-interpretive mode) |
| lpT0 | parallel port 1 (interpretive mode for some Tandy printers) |
| lpT1 | parallel port 2 (interpretive mode for some Tandy printers) |
| lpT2 | parallel port of a monochrome/parallel adapter (intrepretive mode for some Tandy printers) |

*Special Handling For Tandy Printers*

Some printers need special handling for some codes. Using the interpretive mode device file causes the following translations to be made to data being sent to the printer:

| Character | Translates to |
|---|---|
| LF | CR |
| TAB | appropriate number of spaces |
| FF | appropriate number of CRs |

The non-interpretive mode (default) causes every character to be sent to the printer unmodified.

The **lpinit** command asks you if you need "special handling for Tandy Carriage Returns and Line Feeds." If you do not (non-interpretive mode), simply press RETURN in response to the prompt. If you do (interpretive mode), answer "y" to the prompt.

If your printer can be set (dip switches) to treat LF as NEWLINE and CR as CR (without a line feed) and the printer can do its own tabs and form feeds, you should use the non-interpretive mode. If you choose interpretive mode, the system will do translations of CR, FF, and TAB characters being sent to the printer. If you ask for these translations, please note the following:

- You must keep the printer's actual top-of-form in sync with the driver's idea of top-of-form.
- If you run a program that does any non-standard line spacing, such as half-line feeds or 8 lines per inch, the top-of-form will be off in subsequent output.

- If your output contains characters that are not uniformly spaced, the TAB translation may not work properly.

## Files

/dev/lp
/dev/lp0
/dev/lp1
/dev/lp2
/dev/lpT0
/dev/lpT1
/dev/lpT2

## Notes

Tandy has also provided the following model interface programs:

| | |
|---|---|
| Tandy.DMP | interfaces with Tandy DMP printers in Tandy mode. |
| emulator | interfaces with Tandy printers in emulation mode. |
| dumb | interfaces with dumb line printers. |

The system manager should examine these files in the */usr/spool/lp/model* directory to determine what options the interfaces accept with the -o option of **lp.**

## See Also

lp(C), lpadmin(C), lpstat(C), lpinit(C), lpsched(C)

## Name

mem, kmem – Memory image file.

## Description

The **mem** file provides access to the computer's physical memory. All byte addresses in the file are interpreted as memory addresses. Thus, memory locations can be examined in the same way as individual bytes in a file. Note that accessing a nonexistent location causes an error.

The **kmem** file is the same as **mem** except that it corresponds to kernel virtual memory rather than physical memory.

In rare cases, the **mem** and **kmem** files may be used to write to memory and memory-mapped devices. Such patching is not intended for the inexperienced user and may lead to a system crash if not conducted properly. Patching device registers is likely to lead to unexpected results if the device has read-only or write-only bits.

## Files

/dev/mem

/dev/kmem

## Name

messages – Description of system console messages.

## Description

This section describes the various system messages which may appear on the system console. The messages are categorized as follows:

*Fatal*
Recovery is impossible.

*System inconsistency*
A contradictory situation exists in the kernel.

*Abnormal*
A probably legitimate but extreme situation exists.

*Hardware*
Indicates a hardware problem.

*User error*
The user has caused the problem.

Fatal system messages begin with "panic:" and indicate hardware problems or kernel inconsistencies that are too severe for continued operation. After displaying a fatal message, the system will stop. Rebooting is required.

System inconsistency messages indicate problems usually traceable to hardware malfunction, such as memory failure. These messages rarely occur since associated hardware problems are generally detected before such an inconsistency can occur.

Abnormal messages represent kernel operation problems, such as the overflow of critical tables. It takes extreme situations to bring these problems about, so they should never occur in normal system use.

System messages sometime specify the device, *dev*, that caused the error. Each message gives a device specification of the form *nn/mm* where *nn* is the major number of the device, and *mm* is its minor number. The command pipeline

ls -l /dev | grep *nn* | grep *mm*

may be used to list the name of the device associated with the given major and minor numbers.

User errors are situations caused by the user trying to do something contradictory, for example, trying to write to a write-protected floppy.

### Device Driver Messages

Device drivers display error messages in five basic formats, two of which are special and are related to specific drivers. Their formats are as follows:

*Device* **Drive** *n*: *specific error message*
*Device* **Drive** *n*: *type* **error while Reading Cylinder** *cyl* **Head** *hd* **Sector** *sec* **[Status:** *hhhh*]
*Device* **Drive** *n*: *type* **error while Writing Cylinder** *cyl* **Head** *hd* **Sector** *sec* **[Status:** *hhhh*]
*Device* **Drive** *n*: *type* **error while Reading Block** *blk* **[Status:** *hhhh*]
*Device* **Drive** *n*: *type* **error while Writing Block** *blk* **[Status:** *hhhh*]
*Device* **Drive** *n*: *type* **error during** *cmd* **from Block** *blk* **[Status:** *hhhh*]

*Device* **Drive** *n*: *specific error message*

This format is used for general errors from the driver which affect all operations. For example, the message "Cartridge Drive 0: drive not ready" is displayed when the disk cartridge driver tried some I/O operation and the controller reported the drive busy or not ready.

The following messages are possible from both the disk cartridge driver and the tape driver:

Further I/O aborted until device closed
Device closed; error cleared
Drive not ready
Hardware Failure [Status: *hhhh*]
Write protected

These are displayed only by the disk cartridge driver:

active disk changed
active drive not ready
active drive write protected
bad format
bad sector size
SCSI Bus Hung - Resetting
CMD Phase Violation

These are displayed only by the tape driver:

No tape in drive
Drive Hung

These messages are displayed only by the fixed disk driver:

Bad signature
Can't read bad block map
Invalid partition table

And this is displayed only by the floppy driver:

Time Out

**Further I/O aborted until device closed**
**Device closed; error cleared**

These messages are seen when an unrecoverable error has occurred on a device and the driver is refusing to do any more I/O until the device is closed (or until XENIX shuts down). The first usually appears after the error messages describing the unrecoverable error, and the second after the device driver is closed. *Hardware* or *User error.*

**Hardware Failure [Status:** *hhhh*]
This message is displayed when the user tries accessing a device, but when the system was booted the driver was unable to talk to its associated hardware. This could mean that the associated hardware was not installed, or that the hardware was not setup properly. (See the driver's manual page or help file for a description of what the status data is). *Hardware.*

**Drive not ready**
This indicates that when the drive was accessed, the hardware reported the drive was not ready when it should have been. This can happen, for example, if the drive is powered off when the user tries to access it. *Hardware* or *User error.*

**write protected**
This error is displayed when a user tries to write to or mount something and the media is write-protected. *User error.*

**active disk changed**
This error occurs when a disk is changed while it was mounted. This message is very difficult to cause on cartridge drives. *User Error.*

**active drive not ready**
This error occurs when the system tried some operation on a mounted drive, and the controller reported the drive busy or not ready. *Hardware* or *User error.*

**active drive write protected**
This message is seen when the system accesses a previously write-enabled cartridge disk which is mounted, and it is write-protected. This is a very difficult error to cause because while a cartridge is mounted, it is very difficult to remove. *User Error.*

**bad format**
This error occurs when the user tries to access a cartridge which is not formatted or is otherwise not readable. *Hardware* or *User error.*

**bad sector size**
This is a very unusual error. It indicates that when the system looked at a cartridge to open it for I/O, it had an unusual (ie, non-standard) sector size. This error is unusual because there is only one sector size used on the disk cartridge system. *Hardware.*

**Drive hung**
This error indicates that the tape driver was unable to talk to its drive or read its status. *Hardware.*

**No tape in drive**
This message indicates that either there is no tape cartridge in the drive, or that the user removed it during drive operation. *Hardware* or *User error.*

**Bad signature**
The indicated fixed drive may not have been initialized properly. The drive should not be used until both *badtrack*(M) and *fdisk*(M) has been used to initialized the drive. *Hardware* or *User error.*

**Can't read bad block map**
An attempt to read bad block information from the specified fixed drive has failed. This may indicate a hardware malfunction. *Hardware.*

**Time Out**
An attempt to read from or write to the indicated device has failed. The drive door may be open; the media may be bad; or, in the case of a floppy diskette, the disk may have the wrong density for the requested read or write. *Hardware* or *User error.*

**SCSI Bus Hung - Restting**
The SCSI bus, which connects the DCS to your computer, was in a non-idle phase when it should be. Bus Reset is asserted and the operation is retried. *Hardware error.*

**CMD Phase Violation**
The SCSI bus did not enter or exit the command phase when it was supposed to. *Hardware error.*

*Device* **Drive** *n*: *type* **error while Reading Cylinder** *cyl* **Head** *hd* **Sector** *sec* **[Status:** *hhhh*]
*Device* **Drive** *n*: *type* **error while Writing Cylinder** *cyl* **Head** *hd* **Sector** *sec* **[Status:** *hhhh*]
*Device* **Drive** *n*: *type* **error while Reading Block** *blk* **[Status:** *hhhh*]
*Device* **Drive** *n*: *type* **error while Writing Block** *blk* **[Status:** *hhhh*]

This format is used for errors which occur during I/O of some sort. The type of error, some hardware-related status, what type of I/O it occurred on (read or write), and where it occurred is displayed. The *type* of error can be one of the following:

command
hard
no-data
soft
unclassified

A *hard* error indicates that the driver retried the operation and was unable to complete the requested I/O.

*No-data* errors only happen on tape, and indicate that the tape drive was unable to find any recorded data during the requested operation.

A *soft* error indicates that an error occurred, the driver retried the operation and it succeeded. The data is *valid.* These are usually indications that the media is aging.

A *command* error is a rare error indicating that the system made an invalid request of some sort.

An *unclassified* error is an extremely rare error type indicating that the driver was unable to determine what the problem was.

The status displayed (*hhhh*) is data that might be useful from the controller or drive. To see what the displayed stati mean, refer to the driver's manual page or help file.

**panic: cd major number**
This indicates that the system was put together inproperly and the disk cartridge driver was unable to find its major number. *System Inconsistency, Fatal.*

## System Messages

** ABNORMAL System Shutdown **
This message appears when errors occur during normal system shutdown. It is usually accompanied by other system messages. *System inconsistency, fatal.*

bad block on dev *nn/mm*
A nonexistent disk block was found on, or is being inserted in, the structure's free list. *System inconsistency.*

bad count on dev *nn/mm*
A structural inconsistency in the superblock of a file system. The system attempts a repair, but this message will probably be followed by more complaints about this file system. *System inconsistency.*

Bad free count on dev *nn/mm*

A structural inconsistency in the superblock of a file system. The system attempts a repair, but this message will probably be followed by more complaints about this file system. *System inconsistency.*

Can't allocate message buffer

This message appears during XENIX startup if memory cannot be found for the message buffers used by the interprocess communication system calls. This represents an internal system error unless there is an inadequate amount of memory present on the system. XENIX requires that your system have at least 512K memory for reasonable performance. *Abnormal.*

iaddress > 2^24

This indicates an attempted reference to an illegal block number, one so large that it could only occur on a file system larger than 8 billion bytes. *Abnormal.*

Inode table overflow

Each open file requires an inode entry to be kept in memory. When this table overflows the specific request (usually *open*(S) or *creat*(S)) is refused. Although not fatal to the system, this event may damage the operation of various spoolers, daemons, the mailer, and other important utilities. Anomalous results and missing data files are a common result. The process will exit with the error "ENFILE". *Abnormal.*

interrupt from unknown device, vec=*xxxx*

The CPU received an interrupt via a supposedly unused vector. This message is followed by "panic: unknown interrupt." Typically this event comes about when a hardware failure miscomputes the vector of a valid interrupt. *Hardware, Fatal.*

Map overflow (map type indicatory), shutdown and reboot

The map indicated by the map type indicator has fragmented to such an extent that there are not enough map elements to keep track of all the resource pieces. The map is either the core map, the swap map, the message map used in interprocess communications, or the semaphore map. *Abnormal.*

Maxmem was reduced based on the size of the swap area.
Refer to the system documentation for information on
the relationship between memory size and swap size.

*Maxmem* represents the maximum amount of memory available to one user process. It is normally 75% of all available user memory. However, the swap area must be at least as big as *maxmem* , since the largest process allowable must be able to swap out if necessary. If, on startup, the kernel discovers that *maxmem* is larger than the swap area, it will reduce *maxmem* to approximately the size of the swap area. If you get this message when you start up your XENIX system, it indicates that you need to increase the size of the swap partition on your file system in order to make maximum use of the memory available on your machine. This requires backing up the disk, repartitioning the disk, and reloading all file systems on the disk. This should be done with extreme care. *Abnormal.*

For further information on how to increase the size of the swap partition on your file system, please refer to "Altering the Swap Partition Size" section XENIX *Installation Guide.*

In addition, the kernel will have to be relinked with a larger value for the swap area designated in the file *xenixconf* in */usr/sys/conf* . The following line, in *xenixconf* indicates that the swap area is 3000K starting at 6000K on the disk:

swap disk 5 3000

If you increased the size of the swap area, change the 3000 to the size of the new swap are in K-bytes.

*Maxmem* can be tuned to other than the default of 75% of user memory. The default will be used if the configurable parameter "maxprocmem" was set to "O" in the *master* file in */usr/sys/conf* when the version of */xenix* you are running was linked. To specify *maxmem* , define a non–zero value for *maxprocmem* in the *xenixconf* file in */usr/sys/conf* and relink a new version of *xenix* . (The *master* file is all the default configurable parameters and should not be changed; *xenixconf* is the file of exceptions to the defaults.)

*Maxprocmem* is the maximum size of a user process in K. This number can be no larger than 75% of the size of your swap area.

If you decide that the reduced size of *maxmem* (approximately the size of your swap area) is acceptable, you do not need to take any action in response to this message.

See the documentation for *proctl* and *runbig* for discussion on how to run processes that are too big to swap.

no file

There are too many open files, the system has run out of entries in its "open file" table. The warnings given for the message "inode table overflow" apply here. *Abnormal.*

no space on dev *nn/mm*

This message means that the specified file system has run out of free blocks. Although not normally as serious, the warnings discussed for "inode table overflow" apply: often programs are written casually and ignore the error code returned when they tried to write to the disk; this results in missing data and "holes" in data files. The system administrator should keep close watch on the amount of free disk space and take steps to avoid this situation. *Abnormal.*

** Normal System Shutdown **

This message appears when the system has been shutdown properly. It indicates that the machine may now be rebooted or powered down.

Out of device descriptors, increase gdt size (NGDT) and relink XENIX

The number of global descriptor table (gdt) entries that are reserved for device drivers is inadequate to fill the requests of the drivers on the system. A device driver has requested a *gdt* entry and has been refused because the table is full. The value assigned to NGDT in */usr/sys/h/machdep.h* must be increased and the kernel must be relinked after removing the old configuration file, */usr/sys/conf/c.c. Abnormal.*

Out of inodes on dev *nn/mm*

The indicated file system has run out of free inodes. The number of inodes available on a file system is determined when *mkfs*(C) is run. The default number is quite generous, this message should be very rare. The only recourse is to remove some worthless files from that file system, or dump the entire system to a backup device, rerun *mkfs*(C) with more inodes specified, and restore the files from backup. *Abnormal.*

out of text

When programs linked with the *ld* **–i** or **–n** switch are run, a table entry is made so that only one copy of the pure text will be in memory even if there are multiple copies of the program running. This message appears when this table is full. The system refuses to run the program which caused the overflow. Note that there is only one entry in this table for each different pure text program. Multiple copies of one program will not require multiple table entries. Each "sticky" program (see *chmod*(C)) requires a permanent entry in this table; nonsticky pure text programs require an entry only when there is at least one copy being executed. The process will be killed. You can resubmit it at a later, less busy, time. *Abnormal.*

panic: bad 287 int

Attempted execution of a real mode 287 instruction. *System inconsistency, fatal.*

panic: blkdev

An internal disk I/O request, already verified as valid, is discovered to be referring to a nonexistent disk. *System inconsistency, fatal.*

panic: devtab

An internal disk I/O request, already verified as valid, is discovered to be referring to a nonexistent disk. *System inconsistency, fatal.*

panic: iinit

The super-block of the root file system could not be read. This message occurs only at boot time and indicates that a device error occurred and may mean that the root or swap device need repair. *Hardware, fatal.*

panic: IO err in swap

A fatal I/O error occurred while reading or writing the swap area. *Hardware, fatal.*

panic: Kernel buffer crosses 64K boundary, change load address
XENIX cannot have any of its I/O buffers spanning a 64K address boundary. This is checked on system startup and can be corrected by specifying a different XENIX load address to the boot program when you boot XENIX. *Abnormal, fatal.*

panic: memory failure - parity error
A hardware memory failure trap has been taken. If the system contains ECC (Error Correcting Code) memory boards, the message will be slightly different and may indicate the board that caused the error. XENIX will only panic if the error cannot be corrected on the ECC memory board. *System inconsistency, fatal.*

panic: memory management failure
An error occurred during memory management operations. That is, one of the memory management routines encountered an error when doing critical copying or allocating functions. This indicates an internal error and not an equipment malfunction. *System inconsistency, fatal.*

panic: no fs
The device specified in a command is not currently mounted. This error condition has occurred deep in the kernel system call processing and as such should never occur since the possibility has been checked in earlier system code. *System inconsistency, fatal.*

panic: no imt
A mounted file system does not have an entry in the mount table. This should not happen because the mount table is checked by previous code. *System inconsistency, fatal.*

panic: no procs
Each user is limited in the amount of simultaneous processes he can have; an attempt to create a new process when none is available or when the user's limit is exceeded is refused. That is an occasional event and produces no console messages; this panic occurs when the kernel has certified that a free process table entry is available and yet can't find one when it goes to get it. *System inconsistency, fatal.*

panic: Out of swap
There is insufficient space on the swap disk to hold a task. The system refuses to create tasks when it feels there is insufficient disk space. Repeated occurrence of this error indicates that the size of the swap partition size should be increased.

panic: general protection trap
General protection trap taken in kernel. *System inconsistency, fatal.*

panic: preadi
An error occurred when the system was attempting to read in a process' segments during a process switch. This represents an internal system failure. *System inconsistency.*

panic: segment not present
An attempt has been made to access an invalid segment. It may also indicate the segment-not-present trap has been taken in the kernel. *System inconsistency, fatal.*

panic: Timeout table overflow
The timeout table is full. Timeout requests are generated by device drivers, there should usually be room for one entry per system serial line plus ten more for other usages. There is no acceptable behavior that the kernel can provide if this situation arises, except to panic. *System inconsistency, fatal.*

panic: Trap in system
The CPU has generated an illegal instruction trap while executing kernel or device driver code. This message is preceded with an information dump describing the trap. *System inconsistency, fatal.*

panic: Invalid TSS
Internal tables have become corrupted. *System inconsistency, fatal.*

panic: unknown interrupt
The CPU received an interrupt via a supposedly unused vector. Typically this event comes about when a hardware failure miscomputes the vector of a valid interrupt. *Hardware, fatal.*

proc on q
The system attempts to queue a process already on the process ready-to-run queue. *System inconsistency, fatal.*

spurious kb interrupt
This message indicates a serious hardware malfunction associated with the system console. *Hardware.*

Trap *violation number* in USER (or SYSTEM)
This message precedes a "panic:" message and occurs when a process causes an exception or trap on the 286 that cannot be handled in software. The *violation number* is the vector number of the exception. A description of the trap vectors on the 286 can be found in the *iAPX 286 Programmer's Reference Manual* published by Intel.

A trap that occurs while the 286 was executing user code is said to have occurred in USER mode; one that occurs while executing kernel code has occurred in SYSTEM mode. The message is followed by a dump of registers. *System inconsistency, fatal.*

The following is a set of panic messages that may appear:

287 Exception
The 286 thinks it has received a trap from the 287 when there is no 287 on the system.

287 Segment Overrun

A trap type 9 (processor Extension Segment Overrun Exception) has occurred on a system that does not have a 287. This trap indicates that the 287 has overrun the limit of a segment while attempting to read or write the second or subsequent word of an operand.

If a system that does have a 287 generates this exception, it will be killed with a SIGSEGV (segment violation) signal.

bad 287 int

A trap type 7 (Processor Extension Not Available Exception) has occurred on a system that has a 287 chip present. If the system has no 287 on board, the process generating this exception will receive a SIGILL (illegal instruciton) signal and will be killed.

## Name

micnet – The Micnet default commands file.

## Description

The **micnet** file lists the system commands that may be executed through the *remote* command. The file is required for each system in a Micnet network. Whenever a *remote* command is received through the network, the Micnet programs search the **micnet** file for the system command specified with the *remote* command. If found, the command is executed. Otherwise, the command is ignored and an error message is returned to the system which issued the *remote* command.

The file may contain one or more lines. If all commands may be executed, then only the line

executeall

is required in the file. Otherwise, the commands must be listed individually. A line that defines an individual command has the form:

*command=commandpath*

*Command* is the command name to be specified in a *remote* command. *Commandpath* is the full pathname of the command on the specified system. The equal sign (=) separates the command and commandpath. For example, the line

cat=/bin/cat

defines the command name *cat* (used in the *remote* command) to refer to the system command *cat* in the **/bin** directory.

When *executeall* is set, commands are sought in a series of default directories. Initially, the directories are **/bin** and **/usr/bin**. The default directories can be explicitly defined in the file by including a line of the form:

execpath=PATH= *directory*[:*directory*]...

The first part of the line, "execpath=PATH=", is required. Each *directory* must be a valid pathname. The colon is required to separate directories. For example, the line:

execpath=PATH=/bin:/usr/bin:/usr/bobf/bin

sets the default directories to **/bin, /usr/bin,** and **/usr/bobf/bin.**

## Files

/etc/default/micnet

## See Also

aliases(M), cat(C), default(M), netutil(C), remote(C), rep(C), systemid(M), top(M)

## Notes

The *rcp* command cannot be executed from a remote system unless the **micnet** file contains either *executeall* , or the line:

rcp=/usr/bin/rcp

## Name

mt, cmt – Streaming tape drive.

## Description

The *mt* driver provides an interface to the streaming tape device which resembles the classic 9-track tape drive in functionality. However, because the tape is a streaming tape drive, it is recommended that large quantities of data be buffered and then written to the raw interface all at once. The *tdd* program was written for this purpose and can buffer very large amounts of data. Buffering in this manner will prevent the drive from stopping and starting, which is a very slow operation.

The block files access the drive via the system's normal buffering mechanism, but seeking is not possible, since the drive cannot forward or backspace records. Any I/O following such an attempt will be aborted.

The binary representation of the minor device number is encoded *00000rdd,* where *r* is the ''don't rewind'' bit and *dd* is the drive number.

The ''don't rewind'' bit tells the driver not to rewind the tape when the device is closed. This allows one to put multiple volumes of data on a single tape.

The *cmt* device entries access the ''don't rewind'' interface.

## Files

/dev/mt0
/dev/cmt0
/dev/rmt0
/dev/rcmt0

## See Also

tdd(C), rewind(C), skip(C)

## Notes

When using the raw device files, you must align your input and output on 512-byte boundaries.

The tape driver in the kernel expects that its device will share DMA channel 3 with the cartridge disk device. It is mandatory that the jumpers on the tape controller board are positioned correctly for this driver to work properly. The distributed kernel assumes the streaming tape device to use interrupt request line 3.

## Name

null – The null file.

## Description

Data written on a null special file is discarded.

Reads from a null special file always return 0 bytes.

## Files

/dev/null

## Name

passwd – The password file.

## Description

*Passwd* contains the following information for each user:

–Login name

–Encrypted password

–Numerical user ID

–Numerical group ID

–Comment

–Initial working directory

–Program to use as shell

This is an ASCII file. Each field within each user's entry is separated from the next by a colon (:). The comment can contain any desired information. Each user is separated from the next by a NEWLINE. If the password field is null, no password is demanded; if the shell field is null, *sh*(C) is used.

This file resides in the **/etc** directory. Because the passwords are encrypted, the file has general read permission and can be used, for example, to map numerical user IDs to names.

The encrypted password consists of 13 characters chosen from a 64-character alphabet (**.**, **/**, **0–9**, **A–Z**, **a–z**), except when the password is null, in which case the encrypted password is also null. Password aging is in effect for a particular user if his encrypted password in the password file is followed by a comma and a non-null string of characters from the above alphabet. (Such a string must be introduced by the super-user.) The first character of the age denotes the maximum number of weeks for which a password is valid. A user who attempts to log in after his password has expired will be forced to supply a new one. The next character denotes the minimum period in weeks which must expire before the password may be changed. The remaining characters define the week (counted from the beginning of 1970) when the password was last changed. (A null string is equivalent to zero.) The first and second characters must have numerical values in the range 0–63, where the dot (.) is equal to 0 and lowercase "z" is equal to 63. If the numerical value of both characters is 0, the user will be forced to change his password the next time he logs in. If the second character is greater than the first, only the super-user will be able to change the password.

## Files

/etc/passwd

## See Also

a64l(S), getpwent(S), group(M), login(M), passwd(C), pwadmin(C)

## Name

profile – setting up an environment at login time

## Description

If you are running the Bourne shell or the Visual Shell and your login directory contains a file named **.profile**, that file will be read and executed before your session begins; **.profiles** are handy for setting exported environment variables and terminal modes. If the **/etc/profile** file exists, it will be executed for every user before the **.profile**. The following example is typical (except for the comments):

```
# Make some environment variables global
export MAIL PATH TERM
# Set file creation mask
umask 22
# Tell me when new mail comes in
MAIL=/usr/mail/myname
# Add my /bin directory to the shell search sequence
PATH=$PATH:$HOME/bin
# Set terminal type
echo "terminal: \c"
read TERM
case $TERM in
        300)        stty cr2 nl0 tabs; tabs;;
        300s)       stty cr2 nl0 tabs; tabs;;
        450)        stty cr2 nl0 tabs; tabs;;
        hp)         stty cr0 nl0 tabs; tabs;;
        745 | 735)  stty cr1 nl1 –tabs; TERM=745;;
        43)         stty cr1 nl0 –tabs;;
        4014 | tek) stty cr0 nl0 –tabs ff1; TERM=4014; echo "\33;";;
        *)          echo "$TERM unknown";;
esac
```

## Files

$HOME/.profile
/etc/profile

## See Also

env(C), environ(M), login(C), mail(C), sh(C), stty(C), su(C), term(M).

## Name

serial - Serial port interface (tty00 through tty09)

## Description

XENIX allows a maximum of 11 users. This means you may have the console and as many as 10 additional users. To accomplish this XENIX uses the tty*nn* device files to access the serial ports of your computer.

Your computer can support up to two serial adapter cards, and up to two multi-channel adapter cards (each with four serial ports). For further information on the Tandy multi-channel adapter card, please refer to the technical reference documentation which is provided with the board.

You can use your computer's serial ports for connecting terminals, creating networks, connecting modems, and other serial specific communication uses.

Serial lines tty00 and tty01 are specific to the single serial port interface which you would normally find using standard serial/parallel interface boards. These device files are configured to the following ports as shown.

| File | Line | Port |
|---|---|---|
| tty00 | Com 0 | jumper address "0x03F8" |
| tty01 | Com 1 | jumper address "0x02F8" |

The following table defines the pin configurations for the serial ports tty00 and tty01:

| Pin | Description |
|---|---|
| 2 | Transmit Data |
| 3 | Receive Data |
| 6 | Request to Send |
| 7 | Signal Ground |
| 8 | Carrier Detect (Data Set Ready) |
| 20 | Data Terminal Ready |

Serial lines tty02, tty03, tty04 and tty05 are specific to the first multichannel board of your system. Similarly, serial lines tty06, tty07, tty08 and tty09 are specific to the second multi-channel board.

Please refer to the reference documentation which comes with each multi-channel board in order to set the appropriate jumpers which specifies the correct interrupt line and addressing ports for each board.

For information on serial line operation, see tty (M).

## Files

| | |
|---|---|
| /dev/tty00 | /dev/tty01 |
| /dev/tty02 | /dev/tty03 |
| /dev/tty04 | /dev/tty05 |
| /dev/tty06 | /dev/tty07 |
| /dev/tty08 | /dev/tty09 |
| /etc/ttys | /etc/ttytype |
| /etc/gettydefs | |

## See Also

getty(M), tty(M)

## Notes

Although XENIX supports up to 11 users, it is advised that too many users heavily burden the system swapper and the CPU. A good rule of thumb is one user per 512K bytes of available RAM.

Do not enable a serial line if you do not have the respective serial hardware installed. Doing so seriously degrades system performance.

The serial ports operate at most standard XENIX baud rates. Also the ports have a DTE (data terminal equipment) configuration.

## Name

systemid – The Micnet system identification file.

## Description

The **systemid** file contains the machine and site-names for a system in a Micnet or *uucp* network. A *machine-name* identifies a system and distinguishes it from other systems in the same network. A *site-name* identifies the network to which a system belongs and distinguishes the network from other networks in the same chain.

The **systemid** file may contain a *site-name* and up to four different machine names. The file has the form:

[*site-name*]
[*machine-name1*]
[*machine-name2*]
[*machine-name3*]
[*machine-name4*]

The file must contain at least one machine name. The other machine names are optional, serving as alternate names for the same machine. The file must contain a site name if more than one machine name is given or if the network is connected to another through a *uucp* link. The site name, when given, must be on the first line.

Each name can have up to eight letters and numbers but must always begin with a letter. There is never more than one name to a line. A line beginning with a pound sign (#) is considered a comment line and is ignored.

The Micnet network requires one **systemid** file on each system in a network with each file containing a unique set of machine names. If the network is connected to another network through a *uucp* link, then each file in the Micnet network must contain the same site name.

The **systemid** file is used primarily during resolution of aliases. When aliases contain site and/or machine names the name is compared with the names in the file and removed if there is a match. If there is no match, the alias (and associated message, file, or command) is passed on to the specified site or machine for further processing.

## Files

/etc/systemid

## See Also

aliases(M), netutil(C), top(M), uucp(C)

## Name

term – Conventional names for terminals.

## Description

These names are used by certain commands (e.g., *nroff*(CT), *mm*(CT), *man*(CT), *vi*(C)) and are maintained as part of the shell environment (see *sh*(C), *profile*(M), and *environ*(M)) in the variable $TERM:

| | |
|---|---|
| 1520 | Datamedia 1520 |
| 1620 | Diablo 1620 and others using the HyType II printer |
| 1620–12 | same, in 12-pitch mode |
| 2621 | Hewlett-Packard HP2621 series |
| 2631 | Hewlett-Packard 2631 line printer |
| 2631–c | Hewlett-Packard 2631 line printer - compressed mode |
| 2631–e | Hewlett-Packard 2631 line printer - expanded mode |
| 2640 | Hewlett-Packard HP2640 series |
| 2645 | Hewlett-Packard HP264n series (other than the 2640 series) |
| 300 | DASI/DTC/GSI 300 and others using the HyType I printer |
| 300–12 | same, in 12-pitch mode |
| 300s | DASI/DTC/GSI 300s |
| 382 | DTC 382 |
| 382s–12 | same, in 12-pitch mode |
| 3045 | Datamedia 3045 |
| 33 | TELETYPE® Model 33 KSR |
| 37 | TELETYPE Model 37 KSR |
| 40–2 | TELETYPE Model 40/2 |
| 4000A | Trendata 4000A |
| 4014 | Tektronix 4014 |
| 43 | TELETYPE Model 43 KSR |
| 450–12 | same, in 12-pitch mode |
| 735 | Texas Instruments TI735 and TI725 |
| 745 | Texas Instruments TI745 |
| 832 | Anderson Jacobsen 832 |
| dumb | generic name for terminals that lack reverse LINEFEED and other special escape sequences |
| hp | Hewlett-Packard (same as 2645) |
| lp | generic name for a line printer |
| Tal | DASI 450 (same as Diablo 1620) |
| Tlp | Lineprinter with column only reverse line feed |
| tn1200 | General Electric TermiNet 1200 |
| tn300 | General Electric TermiNet 300 |
| TX | TX TRAIN printer |

Up to eight characters, chosen from [–a–z0–9], make up a basic terminal name. Terminal sub-models and operational modes are distinguished by suffixes beginning with a –. Names should generally be based on original vendors, rather than local distributors. A terminal acquired from one vendor should not have more than one distinct basic name.

Commands whose behavior depends on the type of terminal should accept arguments of the form **–T***term* where *term* is one of the names given above; if no such argument is present, such commands should obtain the terminal type from the environment variable $TERM, which, in turn, should contain *term*.

## See Also

environ(M), mm(CT), nroff(CT), profile(M), sh(C), stty(C)

## Notes

The list of terminal names given here is not exhaustive. This list is intended to provide a standard naming convention for the most common terminal types. Programs that should adhere to this naming convention do so with some uncertainty. Not all XENIX facilities support all of these options.

## Name

termcap – Terminal capability database.

## Description

The **/etc/termcap** file is a database describing terminals. This database is used by programs such as *vi*(C) and *curses*(S). Terminals are described in **termcap** by giving a set of capabilities and by describing how operations are performed. Padding requirements and initialization sequences are included in **termcap**.

Entries in **termcap** consist of a number of ':' separated fields. The first entry for each terminal gives the names which are known for the terminal, separated by vertical bar (|) characters. The first name is always 2 characters long for compatibility with older systems. The second name given is the most common abbreviation for the terminal, and the last name given should be a long name fully identifying the terminal. The second name should contain no spaces; the last name may well contain spaces for readability.

## Capabilities

The following is a list of the capabilities that can be defined for a given terminal. In this list, (P) indicates padding may be specified, (P*) indicates that padding may be based on the number of lines affected, and uppercase names indicate XENIX extensions (except for CC).

| Name | Type | Pad? | Description |
|---|---|---|---|
| ae | str | (P) | End alternate character set |
| al | str | (P*) | Add new blank line |
| am | bool | | Terminal has automatic margins |
| as | str | (P) | Start alternate character set |
| bc | str | | BACKSPACE if not CTRL-H |
| BE | str | | Bell character |
| bs | bool | | Terminal can BACKSPACE with CTRL-H |
| BS | str | | Sent by BACKSPACE key (if not bc) |
| bt | str | (P) | Back TAB |
| bw | bool | | BACKSPACE wraps from column 0 to last column |
| CC | str | | Command character in prototype if terminal able to be set |
| cd | str | (P*) | Clear to end of display |
| ce | str | (P) | Clear to end of line |
| CF | str | | Cursor off |
| ch | str | (P) | Like cm but horizontal motion only, line stays same |
| CL | str | | Sent by LEFT key |
| cl | str | (P*) | Clear screen |
| cm | str | (P) | Cursor motion |
| CN | str | | Sent by CANCEL key |
| co | num | | Number of columns in a line |
| CO | str | | Cursor on |
| CR | str | | Sent by RIGHT key |
| cr | str | (P*) | RETURN, (default CTRL-M) |

| | | | |
|---|---|---|---|
| cs | str | (P) | Change scrolling region (vt100), like cm |
| cv | str | (P) | Like ch but vertical only. |
| CW | str | | Sent by CHANGE WINDOW key |
| da | bool | | Display may be retained above |
| db | bool | | Display may be retained below |
| dB | num | | Number of millisec of bs delay needed |
| dC | num | | Number of millisec of cr delay needed |
| dc | str | (P*) | Delete character |
| dF | num | | Number of millisec of ff delay needed |
| DK | str | | Sent by DOWN key (if not kd) |
| DL | str | | Sent by DELETE key |
| dl | str | (P*) | Delete line |
| dm | str | | Delete mode (enter) |
| dN | num | | Number of millisec of nl delay needed |
| do | str | | Down one line |
| dT | num | | Number of millisec of tab delay needed |
| ed | str | | End delete mode |
| EE | str | | Edit mode end |
| EG | num | | Number of chars taken by ES and EE |
| ei | str | | End insert mode; give `:ei=:` if **ic** |
| EN | str | | Sent by END key |
| eo | str | | Can erase overstrikes with a blank |
| ES | str | | Edit mode start |
| ff | str | (P*) | Hardcopy terminal page eject (default CTRL-L) |
| G1 | str | | Upper-right (1st quadrant) corner character |
| G2 | str | | Upper-left (2nd quadrant) corner character |
| G3 | str | | Lower-left (3rd quadrant) corner character |
| G4 | str | | Lower-right (4th quadrant) corner character |
| GD | str | | Down-tick character |
| GE | str | | Graphics mode end |
| GG | num | | Number of chars taken by GS and GE |
| GH | str | | Horizontal bar character |
| GS | str | | Graphics mode start |
| GU | str | | Up-tick character |
| GV | str | | Vertical bar character |
| hc | bool | | Hardcopy terminal |
| hd | str | | Half-line down (forward 1/2 linefeed) |
| HM | str | | Sent by HOME key (if not kh) |
| ho | str | | Home cursor (if no **cm**) |
| HP | str | | Sent by HELP key |
| hu | str | | Half-line up (Reverse HALF-LINEFEED) |
| hz | str | | Hazeltine; can't print ~'s |
| ic | str | (P) | Insert character |
| if | str | | Name of file containing **is** |
| im | bool | | Insert mode (enter); give ':im=:' if **ic** |
| in | bool | | Insert mode distinguishes nulls on display |
| ip | str | (P*) | Insert pad after character inserted |
| is | str | | Terminal initialization string |
| k0-k9 | str | | Sent by 'other' function keys 0-9 |
| kb | str | | Sent by BACKSPACE key |
| kd | str | | Sent by terminal DOWN key |
| ke | str | | Out of 'keypad transmit' mode |

| | | | |
|---|---|---|---|
| KF | str | | Key-click off |
| kh | str | | Sent by HOME key |
| kl | str | | Sent by terminal LEFT key |
| kn | num | | Number of 'other' keys |
| KO | str | | Key-click on |
| ko | str | | Termcap entries for other non-function keys |
| kr | str | | Sent by terminal RIGHT key |
| ks | str | | Put terminal in 'keypad transmit' mode |
| ku | str | | Sent by terminal UP key |
| l0-l9 | str | | Labels on 'other' function keys |
| LD | str | | Sent by LINE DELETE key |
| LF | str | | Sent by LINEFEED key |
| li | num | | Number of lines on screen or page |
| LK | str | | Sent by LEFT key (if not kl) |
| ll | str | | Last line, first column (if no **cm**) |
| ma | str | | Arrow key map, used by vi version 2 only |
| mi | bool | | Safe to move while in insert mode |
| ml | str | | Memory lock on above cursor |
| MN | str | | Sent by minus sign key |
| MP | str | | Multiplan initialization string |
| MR | str | | Multiplan reset string |
| mu | str | | Memory unlock (turn off memory lock) |
| nc | bool | | No correctly working RETURN (DM2500,H2000) |
| nd | str | | Non-destructive SPACEBAR (cursor right) |
| nl | str | (P*) | NEWLINE character (default \n) |
| ns | bool | | Terminal is a CRT but doesn't scroll |
| NU | str | | Sent by NEXT UNLOCKED CELL key |
| os | bool | | Terminal overstrikes |
| pc | str | | Pad character (rather than null) |
| PD | str | | Sent by PAGE DOWN key |
| PL | str | | Sent by PAGE LEFT key |
| PR | str | | Sent by PAGE RIGHT key |
| PS | str | | Sent by plus sign key |
| pt | bool | | Has hardware tabs<br>(may need to be set with **is**) |
| PU | str | | Sent by PAGE UP key |
| RC | str | | Sent by RECALC key |
| RF | str | | Sent by TOGGLE REFERENCE key |
| RK | str | | Sent by RIGHT key (if not kr) |
| RT | str | | Sent by RETURN key |
| se | str | | End stand out mode |
| sf | str | (P) | Scroll forwards |
| sg | num | | Number of blank chars left by so or se |
| so | str | | Begin stand out mode |
| sr | str | (P) | Scroll reverse (backwards) |
| ta | str | (P) | TAB (other than CTRL-I or with padding) |
| TB | str | | Sent by TAB key |
| tc | str | | Entry of similar terminal - must be last |
| te | str | | String to end programs that use **cm** |
| ti | str | | String to begin programs that use **cm** |
| uc | str | | Underscore one char and move past it |
| ue | str | | End underscore mode |

| | | |
|---|---|---|
| ug | num | Number of blank chars left by us or ue |
| UK | str | Sent by UP key (if not ku) |
| ul | bool | Terminal underlines even though it doesn't overstrike |
| up | str | Upline (cursor up) |
| us | str | Start underscore mode |
| vb | str | Visible bell (may not move cursor) |
| ve | str | Sequence to end open/visual mode |
| vs | str | Sequence to start open/visual mode |
| WL | str | Sent by WORD LEFT key |
| WR | str | Sent by WORD RIGHT key |
| xb | bool | Beehive (f1=escape, f2=ctrl C) |
| xn | bool | A NEWLINE is ignored after a wrap (Concept) |
| xr | bool | RETURN acts like **ce** \r \n (Delta Data) |
| xs | bool | Standard out not erased by writing over it (HP 264?) |
| xt | bool | Tabs are destructive, magic so char (Teleray 1061) |

*A Sample Entry*

The following entry describes the Concept–100, and is among the more complex entries in the *termcap* file. (This particular concept entry is outdated, and is used as an example only.)

```
c1|c100|concept100:is=\EU\Ef\E7\E5\E8\El\ENH\EK\E\200\Eo&\200:\
    :al=3*\E^R:am:bs:cd=16*\E^C:ce=16\E^S:cl=2*^L:\
      :cm=\Ea%+ %+ :co#80:dc=16\E^A:dl=3*\E^B:ei=\E\200:\
      :eo:im=\E^P:in:ip=16*:li#24:mi:nd=\E=:\
    :se=\Ed\Ee:so=\ED\EE:ta=8\t:ul:up=\E;:vb=\Ek\EK:xn:
```

Entries may continue onto multiple lines by giving a "\" as the last character of a line; empty fields may be included for readability (as above between the last field on a line and the first field on the next). Capabilities in **termcap** are of three types: Boolean capabilities, which indicate that the terminal has some particular feature; numeric capabilities giving the size of the terminal or the size of particular delays; and string capabilities, which give sequences that are used to perform particular terminal operations.

*Types of Capabilities*

All capabilities have two letter codes. For instance, the fact that the Concept has 'automatic margins' (i.e. an automatic RETURN and LINEFEED when the end of a line is reached) is indicated by the capability **am**. Hence the description of the Concept includes **am**. Numeric capabilities are followed by the character '#' and then the value. Thus **co**, which indicates the number of columns the terminal has, gives the value '80' for the Concept.

Finally, string valued capabilities, such as **ce** (clear to end of line sequence) are given by the two character code, an '=', and then a string ending at the next following ':'. A delay in milliseconds may appear after the '=' in such a capability, and padding characters are supplied by the editor after the remainder of the string is sent to provide this delay. The delay can be either a integer, e.g. '20', or an integer followed by an '*', i.e. '3*'. A '*' indicates that the padding required is proportional to the number of lines affected by the operation, and the amount given is the per-affected-unit padding required. When a '*' is specified, it is sometimes useful to give a delay of the form '3.5' which specifies a delay per unit to tenths of milliseconds.

A number of escape sequences are provided in the string valued capabilities for easy encoding of characters there. A **\E** maps to an ESCAPE character, **^x** maps to a CTRL-X for any appropriate x, and the sequences **\n \r \t \b \f** give a NEWLINE, RETURN, TAB, BACKSPACE, and FORMFEED. Finally, characters may be given as three octal digits after a \, and the characters ^ and \ may be given as \^ and \\. If it is necessary to place a (**:**) in a capability it must be escaped in octal as **\072**. If it is necessary to place a NULL character in a string capability it must be encoded as **\200**. The routines that deal with *termcap* use C strings, and strip the high bits of the output very late so that a **\200** comes out as a **\000** would.

*Preparing Descriptions*

We now outline how to prepare descriptions of terminals. The most effective way to prepare a terminal description is by imitating the description of a similar terminal in *termcap* and to build up a description gradually, using partial descriptions with *ex*(C) to check that they are correct. Be aware that a very unusual terminal may expose deficiencies in the ability of the *termcap* file to describe it or bugs in *ex*. To easily test a new terminal description you can set the environment variable TERMCAP to a pathname of a file containing the description you are working on and the editor will look there rather than in **/etc/termcap**. TERMCAP can also be set to the *termcap* entry itself to avoid reading the file when starting up the editor.

*Basic capabilities*

The number of columns on each line for the terminal is given by the **co** numeric capability. If the terminal is a CRT, then the number of lines on the screen is given by the **li** capability. If the terminal wraps around to the beginning of the next line when it reaches the right margin, then it should have the **am** capability. If the terminal can clear its screen, then this is given by the **cl** string capability. If the terminal can BACKSPACE, then it should have the **bs** capability, unless a BACKSPACE is accomplished by a character other than CTRL-H in which case you should give this character as the **bc** string capability. If it overstrikes (rather than clearing a position when a character is struck over) then it should have the **os** capability.

A very important point here is that the local cursor motions encoded in *termcap* are undefined at the left and top edges of a CRT terminal. The editor will never attempt to backspace around the left edge, nor will it attempt to go up locally off the top. The editor assumes that feeding off the bottom of the screen will cause the screen to scroll up, and the **am** capability tells whether the cursor sticks at the right edge of the screen. If the terminal has switch selectable automatic margins, the **termcap** file usually assumes that this is on, i.e. **am**.

These capabilities suffice to describe hardcopy and CRT terminals. Thus the Model 33 teletype is described as

t3 | 33 | tty33:co#72:os

while the Lear Siegler ADM-3 is described as:

cl | adm3|3|lsi adm3:am:bs:cl=^Z:li#24:co#80

*Cursor addressing*

Cursor addressing in the terminal is described by a **cm** string capability, with *printf*(S)-like escapes (**%x**) in it. These substitute to encodings of the current line or column position, while other characters are passed through unchanged. If the **cm** string is thought of as being a function, then its arguments are the line and then the column to which motion is desired, and the **%** encodings have the following meanings:

| | |
|---|---|
| %d | as in *printf*, 0 origin |
| %2 | like %2d |
| %3 | like %3d |
| %. | like %c |
| %+*x* | adds *x* to value, then %. |
| %>*xy* | if value > *x* adds *y*, no output. |
| %r | reverses order of line and column, no output |
| %i | increments line/column (for 1 origin) |
| %% | gives a single % |
| %n | exclusive or row and column with 0140 (DM2500) |
| %B | BCD (16*(x/10)) + (x%10), no output. |
| %D | Reverse coding (x-2*(x%16)), no output. (Delta Data). |

Consider the HP2645, which, to get to row 3 and column 12, needs to be sent \E&a12c03Y padded for 6 milliseconds. Note that the order of the rows and columns is inverted here, and that the row and column are printed as two digits. Thus its **cm** capability is 'cm=6\E&%r%2c%2Y'. The Microterm ACT-IV needs the current row and column sent preceded by a CTRL-T with the row and column simply encoded in binary, 'cm=^T%.%.'. Terminals which use '%.' need to be able to backspace the cursor (**bs** or **bc**), and to move the cursor up one line on the screen (**up** introduced below). This is necessary because it is not always safe to transmit \t, \n CTRL-D and \r, as the system may change or discard them.

A final example is the LSI ADM-3a, which uses row and column offset by a blank character, thus 'cm=\E=%+ %+ '.

*Cursor motions*

If the terminal can move the cursor one position to the right, leaving the character at the current position unchanged, then this sequence should be given as **nd** (non-destructive space). If it can move the cursor up a line on the screen in the same column, this should be given as **up**. If the terminal has no cursor addressing capability, but can home the cursor (to very upper left corner of screen) then this can be given as **ho**; similarly a fast way of getting to the lower left hand corner can be

given as **ll**; this may involve going up with **up** from the home position, but the editor will never do this itself (unless **ll** does) because it makes no assumption about the effect of moving up from the home position.

*Area clears*

If the terminal can clear from the current position to the end of the line, leaving the cursor where it is, this should be given as **ce**. If the terminal can clear from the current position to the end of the display, then this should be given as **cd**. The editor only uses **cd** from the first column of a line.

*Insert/delete line*

If the terminal can open a new blank line before the line where the cursor is, this should be given as **al**; this is done only from the first position of a line. The cursor must then appear on the newly blank line. If the terminal can delete the line which the cursor is on, then this should be given as **dl**; this is done only from the first position on the line to be deleted. If the terminal can scroll the screen backwards, then this can be given as **sb**, but just **al** suffices. If the terminal can retain display memory above then the **da** capability should be given; if display memory can be retained below then **db** should be given. These let the editor understand that deleting a line on the screen may bring non-blank lines up from below or that scrolling back with **sb** may bring down non-blank lines.

*Insert/delete character*

There are two basic kinds of intelligent terminals with respect to insert/delete character which can be described using *termcap*. The most common insert/delete character operations affect only the characters on the current line and shift characters off the end of the line. Other terminals, such as the Concept 100 and the Perkin Elmer Owl, make a distinction between typed and untyped blanks on the screen, shifting upon an insert or delete only to an untyped blank on the screen which is either eliminated, or expanded to two untyped blanks. You can find out which kind of terminal you have by clearing the screen and then typing text separated by cursor motions. Type 'abc def' using local cursor motions (not spaces) between the 'abc' and the 'def'. Then position the cursor before the 'abc' and put the terminal in insert mode. If typing characters causes the rest of the line to shift rigidly and characters to fall off the end, then your terminal does not distinguish between blanks and untyped positions. If the 'abc' shifts over to the 'def' which then move together around the end of the current line and onto the next as you insert, you have the second type of terminal, and should give the capability **in**, which stands for 'insert null'. If your terminal does something different and unusual then you may have to modify the editor to get it to use the insert mode your terminal defines. No known terminals have an insert mode not falling into one of these two classes.

The editor can handle both terminals that have an insert mode and terminals which send a simple sequence to open a blank position on the current line. Give as **im** the sequence to get into insert mode, or give it an empty value if your terminal uses a sequence to insert a blank position. Give as **ei** the sequence to leave insert mode (give this, with an empty value also if you gave **im** an empty value). Now give as **ic** any sequence needed to be sent just before sending the character to be inserted. Most terminals with a true insert mode will not give **ic**, terminals which send a

sequence to open a screen position should give it here. (Insert mode is preferable to the sequence to open a position on the screen if your terminal has both.) If post insert padding is needed, give this as a number of milliseconds in **ip** (a string option). Any other sequence which may need to be sent after an insert of a single character may also be given in **ip**.

It is occasionally necessary to move around while in insert mode to delete characters on the same line (e.g. if there is a TAB after the insertion position). If your terminal allows motion while in insert mode you can give the capability **mi** to speed up inserting in this case. Omitting **mi** will affect only speed. Some terminals (notably Datamedia's) must not have **mi** because of the way their insert mode works.

Finally, you can specify delete mode by giving **dm** and **ed** to enter and exit delete mode, and **dc** to delete a single character while in delete mode.

*Highlighting, underlining, and visible bells*

If your terminal has sequences to enter and exit standout mode these can be given as **so** and **se** respectively. If there are several kinds of standout mode (such as inverse video, blinking, or underlining – half bright is not usually an acceptable 'standout' mode unless the terminal is in inverse video mode constantly) the preferred mode is inverse video by itself. If the code to change into or out of standout mode leaves one or even two blank spaces on the screen, as the TVI 912 and Teleray 1061 do, this is acceptable, and although it may confuse some programs slightly, it can't be helped.

Codes to begin underlining and end underlining can be given as **us** and **ue** respectively. If the terminal has a code to underline the current character and move the cursor one space to the right, such as the Microterm Mime, this can be given as **uc**. (If the underline code does not move the cursor to the right, give the code followed by a nondestructive space.)

If the terminal has a way of flashing the screen to indicate an error quietly (a bell replacement) then this can be given as **vb**; it must not move the cursor. If the terminal should be placed in a different mode during open and visual modes of *ex*, this can be given as **vs** and **ve**, sent at the start and end of these modes respectively. These can be used to change, e.g., from a underline to a block cursor and back.

If the terminal needs to be in a special mode when running a program that addresses the cursor, the codes to enter and exit this mode can be given as **ti** and **te**. This arises, for example, from terminals like the Concept with more than one page of memory. If the terminal has only memory relative cursor addressing and not screen relative cursor addressing, a one screen-sized window must be fixed into the terminal for cursor addressing to work properly.

If your terminal correctly generates underlined characters (with no special codes needed) even though it does not overstrike, then you should give the capability **ul**. If overstrikes are erasable with a space, then this should be indicated by giving **eo**.

*Keypad*

If the terminal has a keypad that transmits codes when the keys are pressed, this information can be given. Note that it is not possible to handle terminals where the keypad only works in local (this applies, for example, to the unshifted HP 2621 keys). If the keypad can be set to transmit or not transmit, give these codes as **ks** and **ke**. Otherwise the keypad is assumed to always transmit. The codes sent by the LEFT, RIGHT, UP, DOWN, and HOME keys can be given as **kl, kr, ku, kd,** and **kh** respectively. If there are function keys such as f0, f1, ..., f9, the codes they send can be given as **k0, k1, ..., k9**. If these keys have labels other than the default f0 through f9, the labels can be given as **l0, l1, ..., l9**. If there are other keys that transmit the same code as the terminal expects for the corresponding function, such as CLEAR SCREEN, the *termcap* 2 letter codes can be given in the **ko** capability, for example, ':ko=cl,ll,sf,sb:', which says that the terminal has CLEAR, HOME DOWN, SCROLL DOWN, and SCROLL UP keys that transmit the same thing as the **cl, ll, sf,** and **sb** entries.

The **ma** entry is also used to indicate arrow keys on terminals which have single character arrow keys. It is obsolete but still in use in version 2 of vi, which must be run on some minicomputers due to memory limitations. This field is redundant with **kl, kr, ku, kd,** and **kh**. It consists of groups of two characters. In each group, the first character is what an arrow key sends, the second character is the corresponding vi command. These commands are **h** for **kl**, **j** for **kd**, **k** for **ku**, **l** for **kr**, and **H** for **kh**. For example, the mime would be **:ma=^Kj^Zk^Xl:** indicating arrow keys left ( CTRL-H ), down ( CTRL-K ), up ( CTRL-Z ), and right ( CTRL-X ). (There is no HOME key on the mime.)

*Miscellaneous*

If the terminal requires other than a NULL (zero) character as a pad, then this can be given as **pc**.

If tabs on the terminal require padding, or if the terminal uses a character other than CTRL-I to TAB, then this can be given as **ta**.

Hazeltine terminals, which don't allow '~' characters to be printed should indicate **hz**. Datamedia terminals, which echo RETURN-LINEFEED for RETURN and then ignore a following LINEFEED should indicate **nc**. Early Concept terminals, which ignore a LINEFEED immediately after an **am** wrap, should indicate **xn**. If an ERASE-EOL is required to get rid of standout (instead of merely writing on top of it), **xs** should be given. Teleray terminals, where tabs turn all characters moved over to blanks, should indicate **xt**. Other specific terminal problems may be corrected by adding more capabilities of the form **xx**.

Other capabilities include **is**, an initialization string for the terminal, and **if**, the name of a file containing long initialization strings. These strings are expected to properly clear and then set the tabs on the terminal, if the terminal has tabs that can be set. If both are given, **is** will be printed before **if**. This is useful where **if** is **/usr/lib/tabset/std**, but **is** clears the tabs first.

*Similar Terminals*

If there are two very similar terminals, one can be defined as being just like the other with certain exceptions. The string capability **tc** can be given with the name of the similar terminal. This capability must be *last* and the combined length of the two entries must not exceed 1024. Since *termlib* routines search the entry from left to right, and since the tc capability is replaced by the corresponding entry, the capabilities given at the left override the ones in the similar terminal. A capability can be cancelled with **xx@** where *xx* is the capability. For example:

hn|2621nl:ks@:ke@:tc=2621:

This defines a 2621nl that does not have the **ks** or **ke** capabilities, and hence does not turn on the function key labels when in visual mode. This is useful for different modes for a terminal, or for different user preferences.

## Files

| | |
|---|---|
| /etc/termcap | File containing terminal descriptions |

## See Also

curses(S), ex(C), more(C), termcap(S), tset(C), vi(C)

## Notes

*Ex* allows only 256 characters for string capabilities, and the routines in *termcap*(S) do not check for overflow of this buffer. The total length of a single entry (excluding only escaped newlines) may not exceed 1024.

The **ma**, **vs**, and **ve** entries are specific to the *vi* program.

Not all programs support all entries. There are entries that are not supported by any program.

This utility was developed at the University of California at Berkeley and is used with permission.

## Name

terminals – List of supported terminals.

## Description

The following is a partial list of terminals that may be used on your XENIX system. The corresponding *names* can be used to assign the terminal type to the TERM (see *environ*(M)). Note that additional terminals and names are defined in the file **/etc/termcap**.

| Name | Terminal |
|---|---|
| 2621 | HP 2621 |
| 3101 | IBM 3101-10 |
| aaegl | Ann Arbor Ambassador 58 |
| adds25 | Adds Regent 25 |
| adm3 | LSI adm3 |
| adm3a | LSI adm3a |
| ansi | Tandy 3000 |
| c100 | Concept 100 |
| c100rv | c100 Rev. Video |
| datapoint | Datapoint 3360 |
| dg | Data General 6053 |
| dmterm | DeskMate terminal |
| dt100 | Tandy DT-100 terminal |
| dt100w | Tandy DT-100 terminal |
| dw1 | DECwriter I |
| dw2 | DECwriter II |
| h19 | Heathkit h19 |
| h1000 | Hazeltine 1000 |
| h1552 | Hazeltine 1552 |
| h1500 | Hazeltine 1500 |
| h1510 | Hazeltine 1510 |
| h1520 | Hazeltine 1520 |
| h2000 | Hazeltine 200 |
| lisa | Apple Lisa XENIX Console |
| mime | Microterm MI1 |
| omron | Omron 802SAG |
| pt210 | TRS-80 PT-210 printing terminal |
| soroc | Soroc 120 |
| td200 | Tandy 200 |
| tek4023 | Teletron 4023 |
| teletec | Teletec Datascreen |
| ti700 | Texas Instruments Silent 700 |
| ti745 | Texas Instruments Silent 745 |
| trs16 | TRS-80 Model 16 Console |
| trs100 | TRS-80 Model 100 |
| trs600 | TRS-80 Model 600 |
| tvi910 | Televideo 910 |
| vt50 | Digital vt50 |

| | |
|---|---|
| vt50h | Digital vt50h |
| vt52 | Digital vt52 |
| vt1 | Digital vt100 |
| vt100s | Digital vt100 132 columns 14 lines |
| vt100w | Digital vt100 132 columns |
| z19 | Zenith z19 |
| z29 | Zenith z29 |
| zen30 | Zentec 30 |
| zen40 | Zentec 40 |
| zen50 | Zentec 50 (Cobra) |

## Files

/etc/termcap

## Name

termio – General terminal interface.

## Description

All asynchronous communications ports use the same general interface, no matter what hardware is involved. The remainder of this section discusses the common features of this interface.

When a terminal file is opened, it normally causes the process to wait until a connection is established. In practice, users' programs seldom open these files; they are opened by *getty*(M) and become a user's standard input, output, and error files. The very first terminal file opened by the process group leader of a terminal file not already associated with a process group becomes the *control terminal* for that process group. The control terminal plays a special role in handling quit and interrupt signals, as discussed below. The control terminal is inherited by a child process during a *fork*(S). A process can break this association by changing its process group using *setpgrp*(S).

A terminal associated with one of these files ordinarily operates in full-duplex mode. Characters may be typed at any time, even while output is occurring, and are only lost when the system's character input buffers become completely full, which is rare, or when the user has accumulated the maximum allowed number of input characters that have not yet been read by some program. Currently, this limit is 256 characters. When the input limit is reached, all the saved characters are thrown away without notice.

Normally, terminal input is processed in units of lines. A line is delimited by a NEWLINE (ASCII LF) character, an end-of-file (ASCII EOT) character, or an end-of-line character. This means that a program attempting to read will be suspended until an entire line has been typed. Also, no matter how many characters are requested in the read call, at most one line will be returned. It is not, however, necessary to read a whole line at once; any number of characters may be requested in a read, even one, without losing information.

Erase and kill processing is normally done during input. By default, a CTRL-H or BACKSPACE erases the last character typed, except that it will not erase beyond the beginning of the line. By default, a CTRL-U kills (deletes) the entire input line, and optionally outputs a NEWLINE character. Both these characters operate on a keystroke basis, independent of any backspacing or tabbing that may have been done. Both the ERASE and KILL characters may be entered literally by preceding them with the ESCAPE character (\). In this case the ESCAPE character is not read. The ERASE and KILL characters may be changed (see *stty*(C)).

Certain characters have special functions on input. These functions and their default character values are summarized as follows:

INTR (Rubout or ASCII DEL) Generates an *interrupt* signal which is sent to all processes designated with the associated control terminal. Normally, each such process is forced to terminate, but arrangements may be made either to ignore the signal or to receive a trap to an agreed upon location; see *signal*(S).

QUIT (CTRL-\ or ASCII FS) Generates a *quit* signal. Its treatment is identical to the interrupt signal except that, unless a receiving process has made other arrangements, it will not only be terminated but a core image file (called **core**) will be created in the current working directory.

ERASE (CTRL–H or ASCII BS) Erases the preceding character. It will not erase beyond the start of a line, as delimited by a NL, EOF, or EOL character.

KILL (CTRL–U or ASCII NAK) Deletes the entire line, as delimited by a NL, EOF, or EOL character.

EOF (CTRL-D or ASCII EOT) May be used to generate an end-of-file from a terminal. When received, all the characters waiting to be read are immediately passed to the program, without waiting for a NEWLINE and the EOF is discarded. Thus, if there are no characters waiting, which is to say the EOF occurred at the beginning of a line, zero characters will be passed back, which is the standard end-of-file indication.

NL (ASCII LF) Is the normal line delimiter. It cannot be changed or escaped.

EOL (ASCII NULL) Is an additional line delimiter, like NL. It is not normally used.

STOP (CTRL-S or ASCII DC3) Temporarily suspends output. It is useful with CRT terminals to prevent output from disappearing before it can be read. While output is suspended, STOP characters are ignored and not read.

START (CTRL-Q or ASCII DC1) Resumes output which has been suspended by a STOP character. While output is not suspended, START characters are ignored and not read. The START/STOP characters cannot be changed or escaped.

The character values for INTERRUPT (INTR), QUIT, ERASE, KILL, EOF, and EOL may be changed to suit individual tastes. The ERASE, KILL, and EOF characters may be escaped by a preceding backslash (\) character, in which case no special function is carried out.

When the carrier signal from the dataset drops, a "hangup" signal is sent to all processes that have this terminal as the control terminal. Unless other arrangements have been made, this signal causes the processes to terminate. If the hangup signal is ignored, any subsequent read returns with an end-of-file indication. Thus, programs that read a terminal and test for end-of-file can terminate appropriately when hung up on.

When one or more characters are written, they are transmitted to the terminal as soon as previously written characters have finished typing. Input characters are echoed by putting them in the output queue as they arrive. If a process produces characters more rapidly than they can be typed, it will be suspended when its output queue exceeds a given limit. When the queue has drained down to the given threshold, the program is resumed.

Several *ioctl*(S) system calls apply to terminal files. The primary calls use the following structure, defined in the file **<termio.h>**:

```
#define   NCC       8
struct    termio {
          unsigned  short   c_iflag;    /* input modes */
          unsigned  short   c_oflag;    /* output modes */
          unsigned  short   c_cflag;    /* control modes */
          unsigned  short   c_lflag;    /* local modes */
          char              c_line;     /* line discipline */
          unsigned  char    c_cc[NCC];  /* control chars */
};
```

The special control characters are defined by the array *c_cc*. The relative positions and initial values for each function are as follows:

| | | |
|---|---|---|
| 0 | INTR | DEL |
| 1 | QUIT | FS |
| 2 | ERASE | BACKSPACE |
| 3 | KILL | CTRL-U |
| 4 | EOF | EOT |
| 5 | EOL | NUL |
| 6 | Reserved | |
| 7 | SWITCH | |

The *c_iflag* field describes the basic terminal input control:

| | | |
|---|---|---|
| IGNBRK | 0000001 | Ignores break condition |
| BRKINT | 0000002 | Signals interrupt on break |
| IGNPAR | 0000004 | Ignores characters with parity errors |
| PARMRK | 0000010 | Marks parity errors |
| INPCK | 0000020 | Enables input parity check |
| ISTRIP | 0000040 | Strips character |
| INLCR | 0000100 | Maps NL to CR on input |
| IGNCR | 0000200 | Ignores CR |
| ICRNL | 0000400 | Maps CR to NL on input |
| IUCLC | 0001000 | Maps uppercase to lowercase on input |
| IXON | 0002000 | Enables start/stop output control |
| IXANY | 0004000 | Enables any character to restart output |
| IXOFF | 0010000 | Enables start/stop input control |

If IGNBRK is set, the break condition (a character framing error with data all zeros) is ignored, that is, not put on the input queue and therefore not read by any process. Otherwise, if BRKINT is set the break condition will generate an interrupt signal and flush both the input and output queues. If IGNPAR is set, characters with other framing and parity errors are ignored.

If PARMRK is set, a character with a framing or parity error which is not ignored is read as the 3-character sequence: 0377, 0, *X*, where *X* is the data of the character received in error. To avoid ambiguity in this case, if ISTRIP is not set, a valid character of 0377 is read as 0377, 0377. If PARMRK is not set, a framing or parity error which is not ignored is read as the character NULL (0).

If INPCK is set, input parity checking is enabled. If INPCK is not set, input parity checking is disabled. This allows output parity generation without input parity errors.

If ISTRIP is set, valid input characters are first stripped to 7-bits, otherwise all 8-bits are processed.

If INLCR is set, a received NL character is translated into a CR character. If IGNCR is set, a received CR character is ignored (not read). Otherwise if ICRNL is set, a received CR character is translated into a NL character.

If IUCLC is set, a received uppercase alphabetic character is translated into the corresponding lowercase character.

If IXON is set, START/STOP output control is enabled. A received STOP character will suspend output and a received START character will restart output. All START/STOP characters are ignored and not read. If IXANY is set, any input character will restart output that has been suspended.

If IXOFF is set, the system will transmit START characters when the input queue is nearly empty and STOP characters when nearly full.

The initial input control value is all bits clear.

The *c_oflag* field specifies the system treatment of output:

| | | |
|---|---|---|
| OPOST | 0000001 | Post-processes output |
| OLCUC | 0000002 | Maps lowercase to uppercase on output |
| ONLCR | 0000004 | Maps NL to CR-NL on output |
| OCRNL | 0000010 | Maps CR to NL on output |
| ONOCR | 0000020 | No CR output at column 0 |
| ONLRET | 0000040 | NL performs CR function |
| OFILL | 0000100 | Uses fill characters for delay |

| | | |
|---|---|---|
| OFDEL | 0000200 | Fills is DELETE, else NULL |
| NLDLY | 0000400 | Selects NEWLINE delays: |
| NL0 | 0 | |
| NL1 | 0000400 | |
| CRDLY | 0003000 | Selects RETURN delays: |
| CR0 | 0 | |
| CR1 | 0001000 | |
| CR2 | 0002000 | |
| CR3 | 0003000 | |
| TABDLY | 0014000 | Selects Horizontal TAB delays: |
| TAB0 | 0 | |
| TAB1 | 0004000 | |
| TAB2 | 0010000 | |
| TAB3 | 0014000 | Expands TABs to spaces |
| BSDLY | 0020000 | Selects BACKSPACE delays: |
| BS0 | 0 | |
| BS1 | 0020000 | |
| VTDLY | 0040000 | Selects Vertical TAB delays: |
| VT0 | 0 | |
| VT1 | 0040000 | |
| FFDLY | 0100000 | Selects FORMFEED delays: |
| FF0 | 0 | |
| FF1 | 0100000 | |

If OPOST is set, output characters are post-processed as indicated by the remaining flags, otherwise characters are transmitted without change.

If OLCUC is set, a lowercase alphabetic character is transmitted as the corresponding uppercase character. This function is often used in conjunction with IUCLC.

If ONLCR is set, the NL character is transmitted as the CR-NL character pair. If OCRNL is set, the CR character is transmitted as the NL character. If ONOCR is set, no CR character is transmitted when at column 0 (first position). If ONLRET is set, the NL character is assumed to perform the RETURN function; the column pointer will be set to 0 and the delays specified for CR will be used. Otherwise the NL character is assumed to perform the LINEFEED function; the column pointer will remain unchanged. The column pointer is also set to 0 if the CR character is actually transmitted.

The delay bits specify how long transmission stops to allow for mechanical or other movement when certain characters are sent to the terminal. In all cases a value of 0 indicates no delay. If OFILL is set, fill characters will be transmitted for delay instead of a timed delay. This is useful for high baud rate terminals which need only a minimal delay. If OFDEL is set, the fill character is DELETE, otherwise it is NULL.

If a FCRMFEED or Vertical TAB Delay is specified, it lasts for about 2 seconds.

NEWLINE delay lasts about 0.10 seconds. If ONLRET is set, the RETURN delays are used instead of the NEWLINE delays. If OFILL is set, 2 fill characters will be transmitted.

RETURN Delay type 1 is dependent on the current column position, type 2 is about 0.10 seconds, and type 3 is about 0.15 seconds. If OFILL is set, delay type 1 transmits 2 fill characters, and type 2 transmits four fill characters.

Horizontal TAB Delay type 1 is dependent on the current column position. Type 2 is about 0.10 seconds. Type 3 specifies that tabs are to be expanded into spaces. If OFILL is set, 2 fill characters will be transmitted for any delay.

BACKSPACE Delay lasts about 0.05 seconds. If OFILL is set, 1 fill character will be transmitted.

The actual delays depend on line speed and system load.

The initial output control value is all bits clear.

The *c_cflag* field describes the hardware control of the terminal:

| | | |
|---|---|---|
| CBAUD | 0000017 | Baud rate: |
| B0 | 0 | Hang up |
| B50 | 0000001 | 50 baud |
| B75 | 0000002 | 75 baud |
| B110 | 0000003 | 110 baud |
| B134 | 0000004 | 134.5 baud |
| B150 | 0000005 | 150 baud |
| B200 | 0000006 | 200 baud |
| B300 | 0000007 | 300 baud |
| B600 | 0000010 | 600 baud |
| B1200 | 0000011 | 1200 baud |
| B1800 | 0000012 | 1800 baud |
| B2400 | 0000013 | 2400 baud |
| B4800 | 0000014 | 4800 baud |
| B9600 | 0000015 | 9600 baud |
| EXTA | 0000016 | External A |
| EXTB | 0000017 | External B |
| | | |
| CSIZE | 0000060 | Character size: |
| CS5 | 0 | 5 bits |
| CS6 | 0000020 | 6 bits |
| CS7 | 0000040 | 7 bits |
| CS8 | 0000060 | 8 bits |
| CSTOPB | 0000100 | Sends two stop bits, else one |
| CREAD | 0000200 | Enables receiver |
| PARENB | 0000400 | Parity enable |
| PARODD | 0001000 | Odd parity, else even |
| HUPCL | 0002000 | Hangs up on last close |
| CLOCAL | 0004000 | Local line, else dial-up |

The CBAUD bits specify the baud rate. The zero baud rate, B0, is used to hang up the connection. If B0 is specified, the data-terminal-ready signal will not be asserted. Without this signal, the line is disconnected if connected through a modem. For any particular hardware, impossible speed changes are ignored.

The CSIZE bits specify the character size in bits for both transmission and reception. This size does not include the parity bit, if any. If CSTOPB is set, 2 stop bits are used, otherwise 1 stop bit. For example, at 110 baud, 2 stops bits are required.

If PARENB is set, parity generation and detection is enabled and a parity bit is added to each character. If parity is enabled, the PARODD flag specifies odd parity if set, otherwise even parity is used.

If CREAD is set, the receiver is enabled. Otherwise no characters will be received.

If HUPCL is set, the line will be disconnected when the last process with the line open closes it or terminates. That is, the data-terminal-ready signal will not be asserted.

If CLOCAL is set, the line is assumed to be a local, direct connection with no modem control. The data-terminal-ready and request-to-send signals are asserted, but incoming modem signals are ignored. If CLOCAL is not set, modem control is assumed. This means the data-terminal-ready and request-to-send signals are asserted. Also, the carrier-detect signal must be returned before communications can proceed.

The initial hardware control value after open is B9600, CS8, CREAD, HUPCL.

The *c_lflag* field of the argument structure is used by the line discipline to control terminal functions. The basic line discipline (0) provides the following:

| | | |
|---|---|---|
| ISIG | 0000001 | Enable signals |
| ICANON | 0000002 | Canonical input (erase and kill processing) |
| XCASE | 0000004 | Canonical upper/lower presentation |
| ECHO | 0000010 | Enables echo |
| ECHOE | 0000020 | Echoes erase character as BACKSPACE-SPACEBAR-BACKSPACE |
| ECHOK | 0000040 | Echoes NL after kill character |
| ECHONL | 0000100 | Echoes NL |
| NOFLSH | 0000200 | Disables flush after interrupt or quit |
| XCLUDE | 0100000 | Exclusive use of the line |

If ISIG is set, each input character is checked against the special control characters INTR and QUIT. If an input character matches one of these control characters, the function associated with that character is performed. If ISIG is not set, no checking is done. Thus these special input functions are possible only if ISIG is set. These functions may be disabled individually by changing the value of the control character to an unlikely or impossible value (e.g., 0377).

If ICANON is set, canonical processing is enabled. This enables the erase and kill edit functions, and the assembly of input characters into lines delimited by NL, EOF, and EOL. If ICANON is not set, read requests are satisfied directly from the input queue. A read will not be satisfied until at least VMIN characters have been received or the timeout value VTIME has expired. This allows fast bursts of input to be read efficiently while still allowing single character input. The VMIN and VTIME values are stored in the position for the EOF and EOL characters respectively. The time value represents tenths of seconds.

If XCASE is set, and if ICANON is set, an uppercase letter is accepted on input by preceding it with a \ character, and is output preceded by a \ character. In this mode, the following escape sequences are generated on output and accepted on input:

| For: | Use: |
|---|---|
| ` | \' |
| \| | \! |
| ~ | \^ |
| { | \( |
| } | \) |
| \ | \\ |

For example, **A** is input as **\a**, **\n** as **\\n**, and **\N** as **\\\n**.

If ECHO is set, characters are echoed as received.

When ICANON is set, the following echo functions are possible. If ECHO and ECHOE are set, the ERASE character is echoed as ASCII BS SP BS, which will clear the last character from a CRT screen. If ECHOE is set and ECHO is not set, the ERASE character is echoed as ASCII SP BS. If ECHOK is set, the NL character will be echoed after the KILL character to emphasize that the line will be deleted. Note that an ESCAPE character preceding the ERASE or KILL character removes any special function. If ECHONL is set, the NL character will be echoed even if ECHO is not set. This is useful for terminals set to local echo (so-called half duplex). Unless escaped, the EOF character is not echoed. Because EOT is the default EOF character, this prevents terminals that respond to EOT from hanging up.

If NOFLSH is set, the normal flush of the input and output queues associated with the QUIT and INTERRUPT characters will not be done.

If XCLUDE is set, any subsequent attempt to open the TTY device using *open*(S) will fail for all users except the super-user. If the call fails, it returns EBUSY in ERRNO. XCLUDE is useful for programs which must have exclusive use of a communications line. It is not intended for the line to the program's controlling terminal. XCLUDE must be cleared before the setting program terminates, otherwise subsequent attempts to open the device will fail.

The initial line discipline control value is all bits clear.

The primary *ioctl*(S) system calls have the form:

ioctl (*fildes, command, arg*)
struct termio **arg*;

The commands using this form are:

TCGETA Gets the parameters associated with the terminal and stores them in the *termio* structure referenced by *arg*.

TCSETA Sets the parameters associated with the terminal from the structure referenced by *arg*. The change is immediate.

TCSETAW Waits for the output to drain before setting the new parameters. This form should be used when changing parameters that will affect output.

TCSETAF Waits for the output to drain, then flushes the input queue and sets the new parameters.

Additional *ioctl*(S) calls have the form:

ioctl (*fildes, command, arg*)
int *arg*;

The commands using this form are:

TCSBRK Waits for the output to drain. If *arg* is 0, it then sends a break (zero bits for 0.25 seconds).

TCXONC Starts/stops control. If *arg* is 0, it suspends output; if 1, it restarts suspended output.

TCFLSH If *arg* is 0, it flushes the input queue; if 1, it flushes the output queue; if 2, it flushes both the input and output queues.

## Files

/dev/tty
/dev/tty*
/dev/console

## See Also

fork(S), ioctl(S), setpgrp(S), signal(S), stty(C), tty(M)

## Name

top, top.next – The Micnet topology files.

## Description

These files contain the topology information for a Micnet network. The topology information describes how the individual systems in the network are connected and what path a message must take from one system to reach another. Each file contains one or more lines of text. Each line of text defines a connection or a communication path.

The **top** file defines connections between systems. Each line lists the machine names of the connected systems, the serial lines used to make the connection, and the speed (baud rate) of transmission between the systems. Each line has the form:

*machine1 tty1 machine2 tty2 speed*

*Machine1* and *machine2* are the machine names of the respective systems as given in the **systemid** files. *Tty1* and *tty2* are the device names (e.g., tty01) of the connecting serial lines. The speed must be an acceptable baud rate (e.g., 110, 300, ..., 19200).

The **top.next** file contains information about how to reach a particular system from a given system. There may be several lines for each system in the network. Each line lists the machine name of a system, followed by the machine name of a system connected to it, followed by the machine names of all the systems that may be reached by going through the second system. Such a line has the form:

*machine1 machine2 machine3* [*machine4*]...

The machine names must be the names of the respective systems as given by the first machine name in the **systemid** files.

The **top.next** file must be present even if there are only two computers in the network. In such a case, the file must be empty.

In the **top** and **top.next** files, any line beginning with a number sign (#) is considered a comment and is ignored.

## Files

/usr/lib/mail/top

/usr/lib/mail/top.next

## See Also

aliases(M), netutil(C), systemid(M), top(M)

## Name

tty – Special terminal interface.

## Description

The **/dev/tty** file is, in each process, a synonym for the control terminal associated with the process group of that process, if any. It is useful for programs or shell sequences that wish to be sure of writing messages on the terminal no matter how output has been redirected. It can also be used for programs that demand the name of a file for output, or when typed output is desired and it is tiresome to find out what terminal is currently in use.

The general terminal interface is described in *termio*(M).

## Files

/dev/tty
/dev/tty*

## See Also

termio(M)

## Name

ttys – Login terminals file.

## Description

The **/etc/ttys** file contains a list of the device special files associated with possible login terminals, and defines which files are to be opened by the *init*(M) program on system start-up.

The file contains one or more entries of the form:

*state mode name*

The *name* must be the filename of a device special file. Only the filename may be supplied, the path is assumed to be **/dev**. If *state* is "1", the file is enabled for logins; if "0", the file is disabled. The *mode* is used as an argument to the *getty*(M) program. It defines the line speed and type of device associated with the terminal. A list of arguments is provided in *getty*(M).

For example, the entry "16tty02" means the serial line tty02 is to be opened for logging in at 9600 baud.

## Files

/etc/ttys

## See Also

disable(C), enable(C), getty(M), gettydefs(F), init(M)

## Notes

The **/etc/ttys** file may only be edited when the system is in system maintenance mode. See the XENIX *Operations Guide*.

## Bugs

The character delay algorithm of the line-discipline output routine has not been implemented in the most effecient manner.

# Index - Miscellaneous (M)